Effect of Different Learning Rates in Batch Gradient Descent

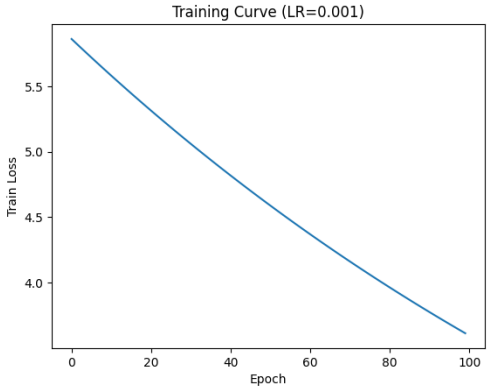
(Implementation/Code file has been attached with the submission)

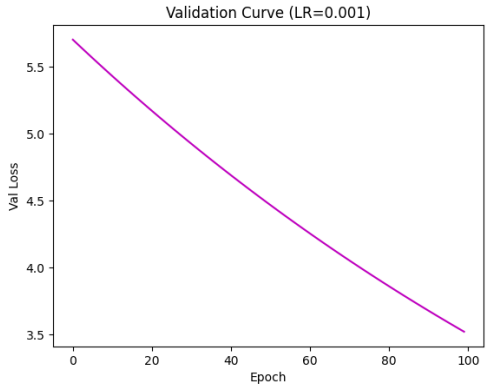
Following Learning rates were used for the experiment:

0.001, 0.01, 0.05, 0.1, and 0.5

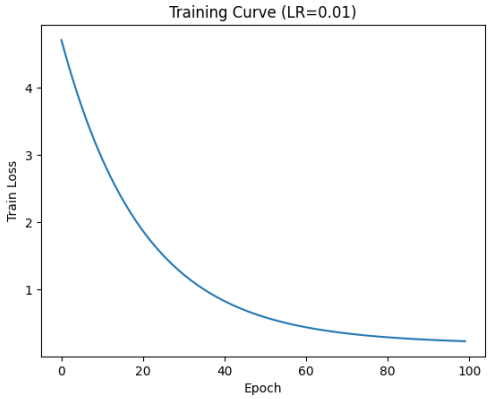
Observations:

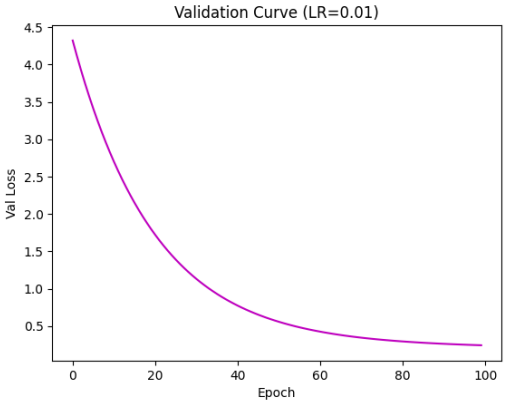
- **Learning rate = 0.001**
  The loss decreased very slowly. The model took many epochs to improve. This learning rate is too small.



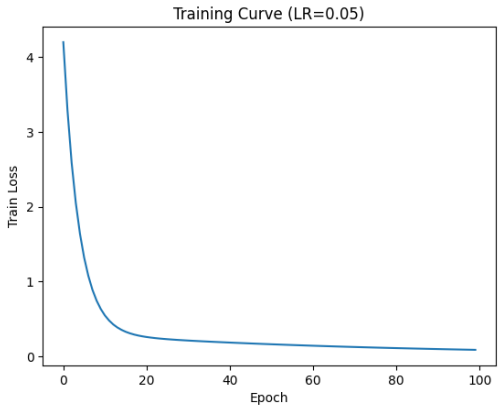


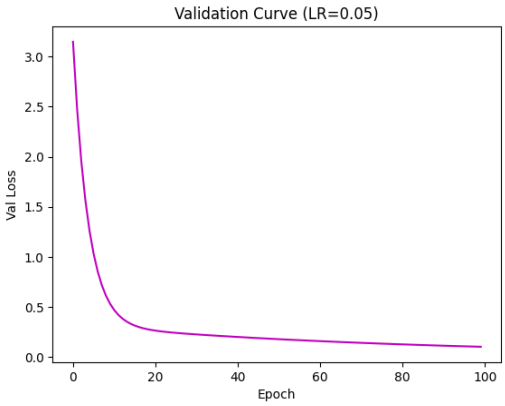
- **Learning rate = 0.01**
  The loss decreased smoothly but still required more epochs to converge.



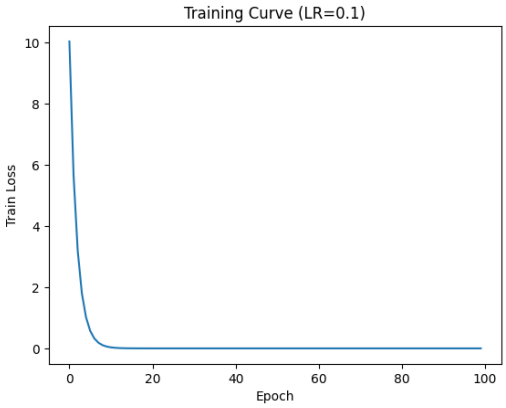


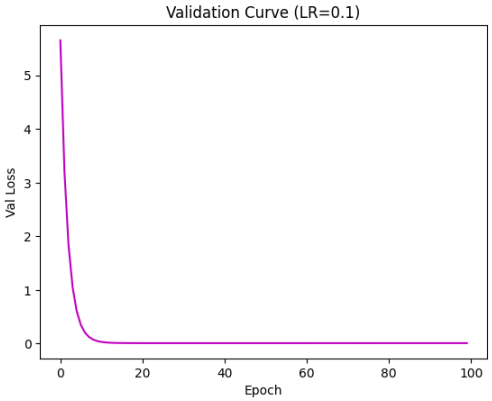
- **Learning rate = 0.05**
  The model converged faster and remained stable. This learning rate gave the best performance.





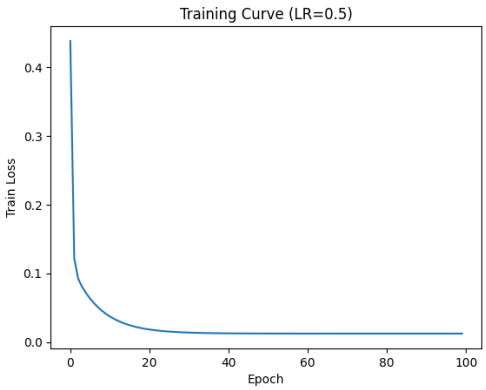
- **Learning rate = 0.1**
  The model converged very quickly with small oscillations.

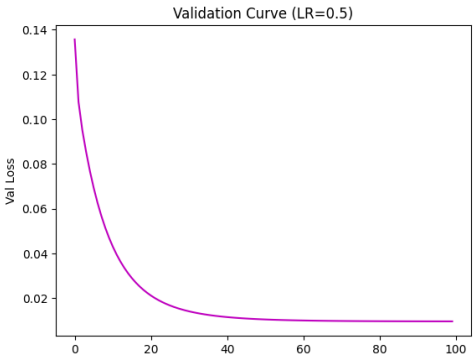




\

- **Learning rate = 0.5**
  The loss fluctuated and sometimes increased. This shows the learning rate is too large and causes instability.





Conclusion:

A moderate learning rate (around **0.05 to 0.1**) gives the best trade-off between speed and stability.

Comparison - Batch vs. Stochastic vs. Mini-Batch

(Implementation/Code file has been attached)

Each method was Implemented using five different learning rates **(0.001,0.01,0.05,0.1,0.5)** and observed distinct differences in their learning curves and convergence behavior.

1. **Batch Gradient Descent**

When running the original lecture code, It was observed that the weight updates occurred only once per epoch. The code calculated yhat using the entire x_train array in a single vectorized operation, and the gradients d b and dw were averages of the errors across all training samples.

- **Visual Observation:** The "Training Loss Vs Epoch" plot produced a **very smooth curve**. Because the gradient is an average of the entire dataset, the path toward the minimum was consistent and stable.
- **Learning Rate Sensitivity:** At lower learning rates (0.001), the loss decreased very slowly, taking many epochs to flatten out. At the highest learning rate (0.5), the curve dropped sharply but remained stable because the noise was averaged out before the update.

2. **Stochastic Gradient Descent (SGD)**

For this experiment, Loop was modified to update weights w and b for **every single training example** individually, rather than calculating the mean error of the whole set at once.

- **Visual Observation:** In contrast to Batch GD, the learning curves for SGD appeared **noisy and jagged**. It was noticed the loss didn't decrease monotonically; it fluctuated up and down slightly as it descended. This is because the code was reacting to the random noise (ϵ) present in individual data points rather than the overall trend.
- **Convergence Speed:** I observed that the loss dropped extremely quickly in the very first epoch. Since there are N=80 training samples, SGD performed 80 updates in a single epoch, whereas Batch GD only performed one. However, the code execution time per epoch was slower due to the lack of vectorization.
- **Instability:** At the high learning rate (0.5), the SGD loss curve oscillate significantly. The high learning rate combined with noisy individual samples caused the weights to "jump" around the minimum rather than settling politely.
-

3. **Mini-Batch Gradient Descent**

It was implemented by splitting the training data into small batches (size 16) and performing updates based on the average error of those batches.

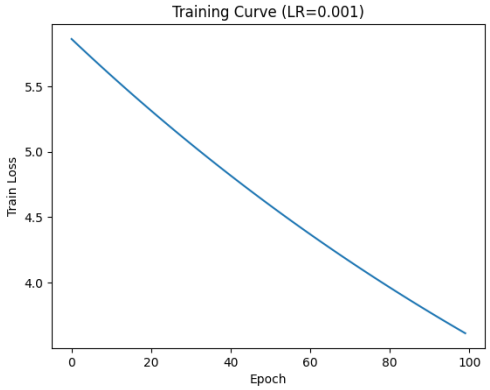
- **Visual Observation:** The results were a hybrid of the previous two. The loss curves were smoother than SGD but still had slight fluctuations compared to the perfectly smooth Batch GD line.
- **Comparison:** This method seemed to offer the best balance. It utilized the vectorized operations for speed (like Batch) but updated the parameters frequently enough to converge faster than Batch GD in terms of epochs.

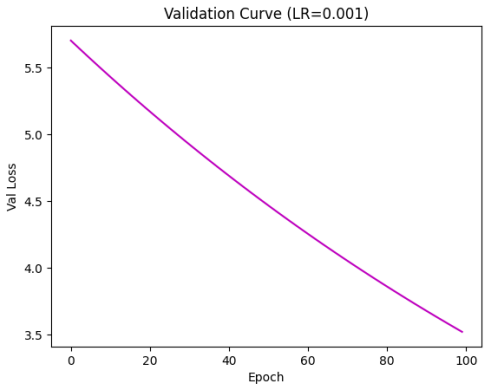
- **Conclusion**

By comparing the train_Losses and val_Losses plots across all three methods, It is concluded that:

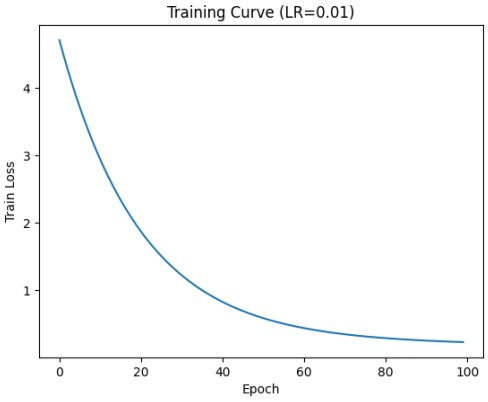
1. **Batch GD** provides the smoothest, most stable trajectory but requires more epochs to converge if the learning rate is small.

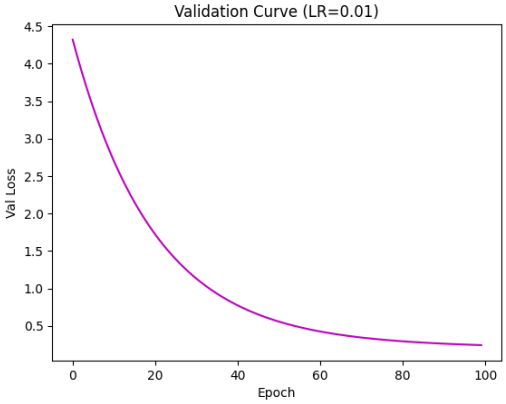
   - **Learning rate = 0.001**
     The loss decreased very slowly. The model took many epochs to improve. This learning rate is too small.



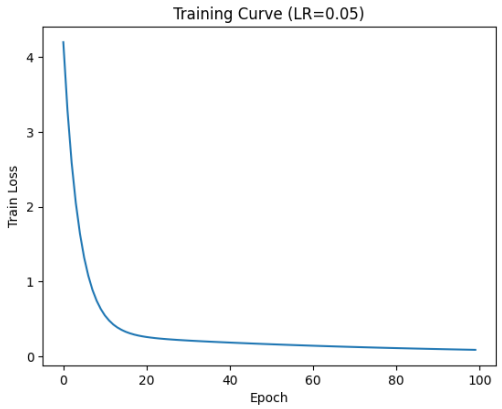


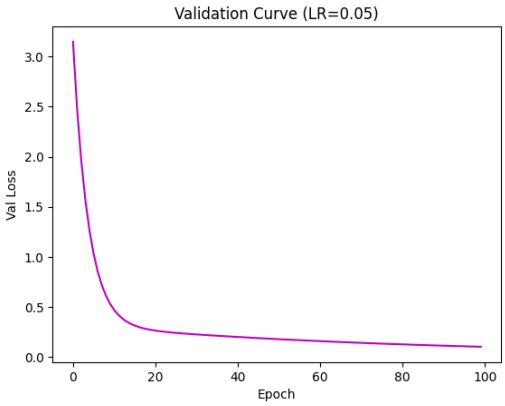
   - **Learning rate = 0.01**
     The loss decreased smoothly but still required more epochs to converge.



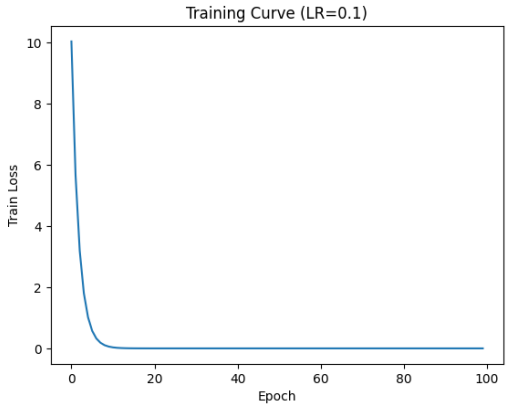


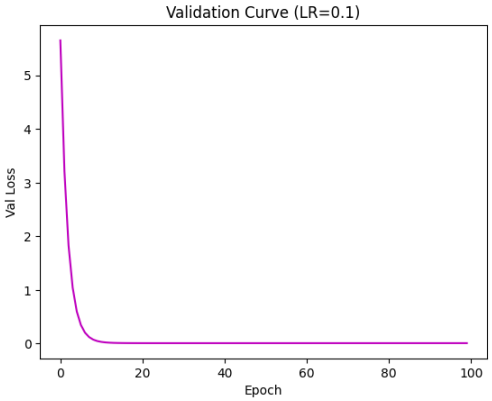
   - **Learning rate = 0.05**
     The model converged faster and remained stable. This learning rate gave the best performance.





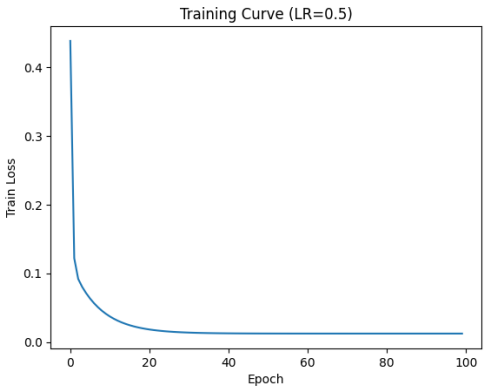
- **Learning rate = 0.1**
  The model converged very quickly with small oscillations.

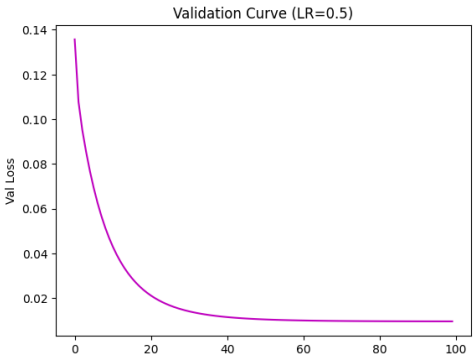




\

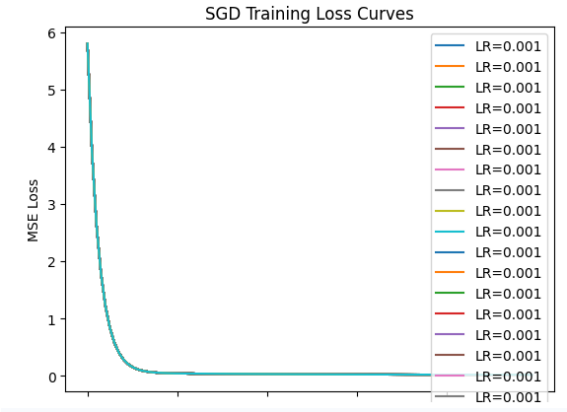
- **Learning rate = 0.5**
  The loss fluctuated and sometimes increased. This shows the learning rate is too large and causes instability.



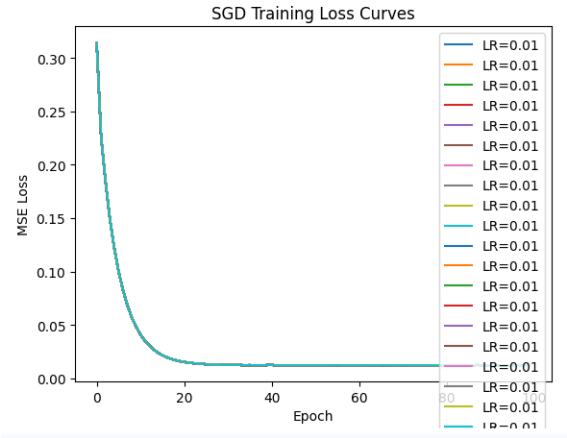


2. **SGD** converges the fastest in terms of "progress per epoch" but produces the noisiest curves and is risky with high learning rates.

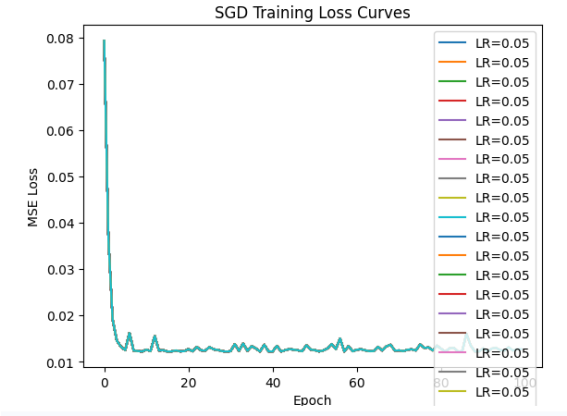
Learning rate = 0.001



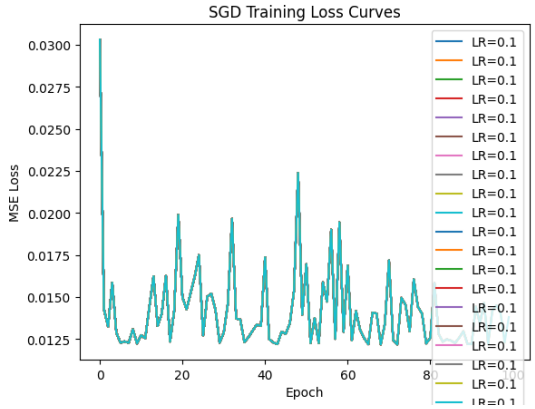
Learning rate = 0.01



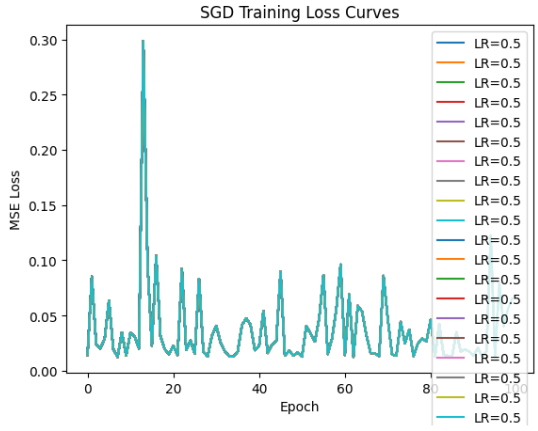
Learning rate = 0.05



Learning rate = 0.1

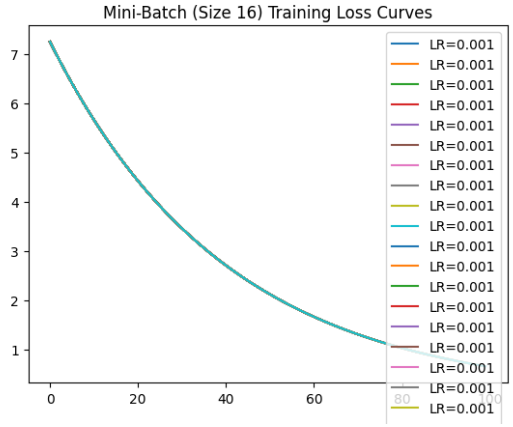


Learning rate = 0.5

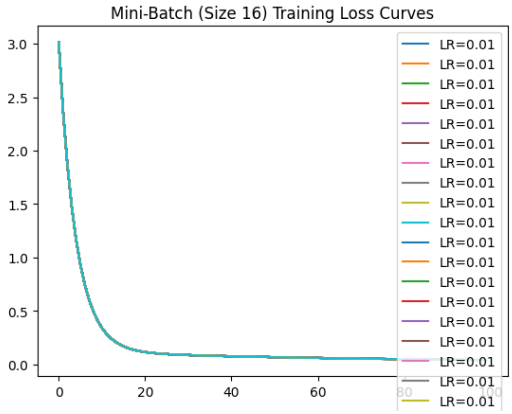


3. **Mini-Batch** provides a compromise, stabilizing the erratic nature of SGD while retaining the computational benefits of the vectorization found in the original source code.

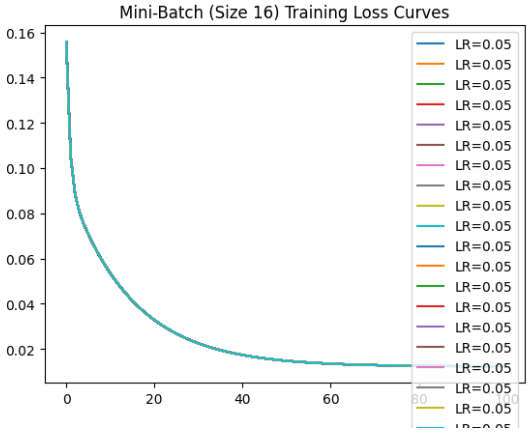
Learning rate = 0.001



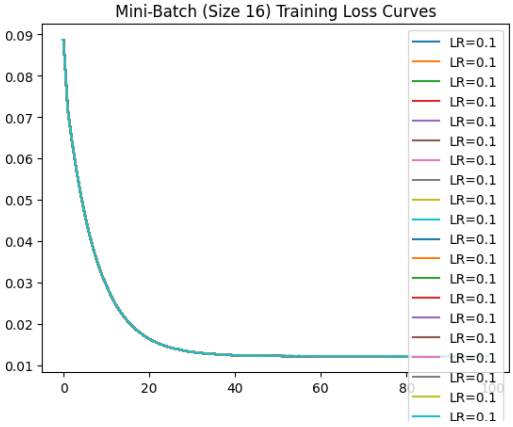
Learning rate = 0.01



Learning rate = 0.05



Learning rate = 0.1



Learning rate = 0.5